

UNIVERSITÀ
DI TORINO

DIPARTIMENTO DI
INFORMATICA

CORSO DI LAUREA
TRIENNALE

A.A. 2026/27

Canale B

Programmazione I

Teoria

Elvio Amparore

Un primo algoritmo

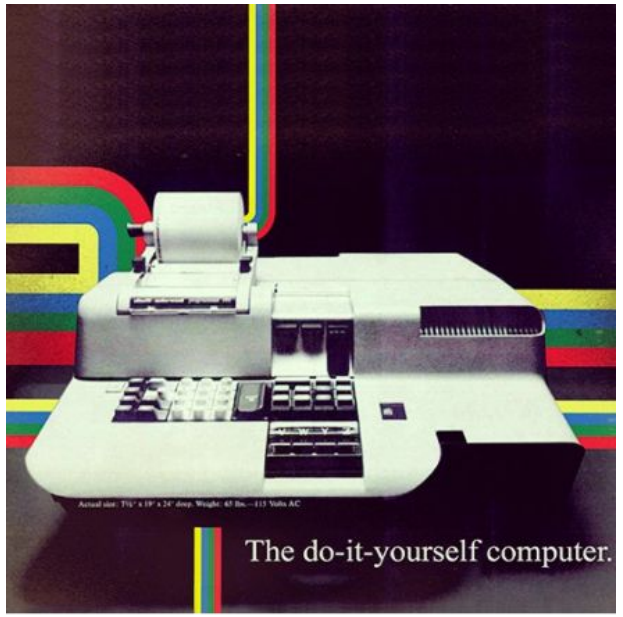
Introduzione passo-passo al concetto di algoritmo

Vocabolario

Calcolatore (IT) / Computer (EN)

- Originariamente persona che eseguiva calcoli manualmente, con l'ausilio di strumenti come carta e penna o calcolatrici meccaniche
- Dagli anni '50, un dispositivo elettromeccanico programmabile al fine di svolgere calcoli anche complessi





Programma 101, 1965

Che cosa non è l'Informatica

L'informatica non è la scienza dei calcolatori. Non più di quanto l'astronomia sia la scienza dei telescopi o la chirurgia la scienza dei bisturi.

– Edsger W. Dijkstra



L'informatica studia gli algoritmi, non i dispositivi.

Che cos'è l'Informatica

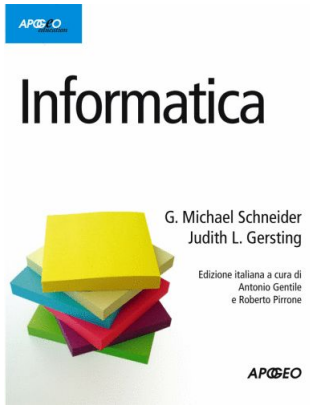
Informatica = lo studio degli algoritmi, che comprende:

- Le loro proprietà formali e matematiche: studiare algoritmi corretti, efficienti
- Le loro realizzazioni fisica: progettare e realizzare computer per eseguire gli algoritmi
- Le loro realizzazioni linguistiche: progettare linguaggi di programmazione per scrivere eseguire gli algoritmi
- Le loro applicazioni: progettare applicazioni software (algoritmi *implementati*) per problemi importanti

Vocabolario: Algoritmo

- Un procedimento di calcolo ben definito eseguibile meccanicamente da un essere umano o da una macchina per ottenere un dato risultato (output) a partire da certi dati di partenza (input)
- Prende il nome dal matematico persiano Muhammad al-Khwarismi (vissuto intorno all'anno 800), il cui libro "*Calcoli con i numerali indiani*" descrive algoritmi di calcolo per le operazioni aritmetiche con i numeri indiani.





- **Algoritmo**: Un insieme ordinato di operazioni non ambigue ed effettivamente computabili che, quando eseguito, produce un risultato e si arresta in un tempo finito

In informatica tutto è numero

Prima di parlare di programmi, fissiamo un'idea fondamentale:

ogni informazione che un computer manipola
è rappresentata come numero.

- Testi, immagini, suoni, istruzioni e dati diversi sembrano oggetti molto differenti, ma nella memoria del computer sono tutti codificati come numeri, precisamente come sequenze di bit (numeri in formato binario).
- Un programma imperativo lavora modificando valori numerici memorizzati nello stato della macchina.
- Programmare significa descrivere come questi valori devono essere trasformati passo dopo passo.

Programmazione imperativa

- Nella **programmazione imperativa** il programma è composto da una *sequenza ordinata* di *istruzioni al computer*, nella forma di un'istruzione per riga (una riga = una istruzione)
- Terminata l'esecuzione della riga *n*-esima, viene eseguita l'istruzione della riga (*n*+1)-esima se non specificato diversamente
- Esempi di istruzioni per la macchina
  - assegna un valore ad una variabile
  - somma due variabili e memorizza il risultato in una terza variabile
  - confronta due variabili
  - salta ad una riga diversa dalla successiva (sotto una certa condizione)

Progettare un algoritmo

- Consideriamo il problema di calcolare la moltiplicazione fra due numeri interi **m** ed **n** ≥ 0.
- Dobbiamo definire:
  - **Input**: i due valori **m** ed **n**
  - **Output**: il valore **m** x **n**
  - **Passi**: una sequenza non ambigua di operazioni che la macchina è in grado di eseguire
  - **Terminazione**: la sequenza deve raggiungere l'output e arrestarsi dopo un numero finito di passi
- Assumiamo che la nostra macchina non disponga direttamente della moltiplicazione, ma sappia eseguire operazioni elementari come somma, assegnamento e confronto.
- Progettare l'algoritmo significa quindi costruire il risultato richiesto usando soltanto queste operazioni disponibili.

Algoritmo per moltiplicazione tra interi

- Riconduciamo la moltiplicazione a un'operazione che la nostra ipotetica macchina sa eseguire: l'addizione.
- Per n ≥ 0 vale

$$m \times n = \underbrace{m + m + \ldots + m}_{\text{Sommo } m \text{ a sé stesso } n-1 \text{ volte}}$$

- Possiamo scrivere la stessa operazione partendo dall'elemento neutro della somma:

$$s = \underbrace{0}_{\text{elemento neutro}} + \underbrace{m + m + \ldots + m}_{\text{Sommo } n \text{ volte } m \text{ all'elemento neutro } 0}$$

- Resta da stabilire come eseguire meccanicamente esattamente **n** addizioni e poi fermarsi.

Algoritmo per moltiplicazione tra interi



parziale iniziale → $s=0$

- Esempio per $m = 4$ e $n = 3$
- Definisco la somma parziale

  $s = 0$

- Tengo traccia delle addizioni già effettuate con le dita della mano



$s = 0$

Algoritmo per moltiplicazione tra interi

- Esempio per ***m*** = 4 e ***n*** = 3
- Dopo la 1° addizione, s vale 4.

```
  0  +
  4  =
------
```

1° parziale → $s = 4$



Ho completato una addizione →



$$s = 0 + 4$$

Algoritmo per moltiplicazione tra interi

- Esempio per ***m*** = 4 e ***n*** = 3
- Dopo la 2° addizione, s vale 8.

```
    0  +
    4  =
  ------
1° parziale →  4  +
    4  =
  ------
2° parziale →  s=8
```

Ho completato due addizioni





$s = 0 + 4 + 4$

Algoritmo per moltiplicazione tra interi

- Esempio per ***m*** = 4 e ***n*** = 3
- Dopo la 3° addizione, s vale 12.

Ho completato tre addizioni e mi fermo



$s = 0 + 4 + 4 + 4$

```
                0  +
                4  =
            ------
1° parziale →   4  +
                4  =
            ------
2° parziale →   8  +
                4  =
            ------
3° parziale
e totale →    s=12
```



Algoritmo per moltiplicazione tra interi

- La moltiplicazione è realizzata sommando ripetutamente il valore *m* ad una somma parziale *s*, inizialmente uguale a 0, per *n* volte
- Per poter eseguire questo algoritmo in modo meccanico dobbiamo disporre di due *strumenti*:
  - un ***accumulatore*** *s* per tenere traccia della somma parziale (equivalente del foglio di carta su cui faccio le addizioni)
  - un ***contatore*** *i* per tenere traccia del numero di addizioni (equivalente al numero di dita della mano usate per contare le ripetizioni)

Due concetti essenziali della programmazione, su cui torneremo spesso:

1. **DATI**: i valori su cui il programma opera, e il modo in cui vengono organizzati.
2. **PROCEDURE**: le operazioni e i procedimenti che trasformano questi dati.

Questa idea si può sintetizzare con la frase (di *Niklaus Wirth*):

Programma = **Algoritmi** + **Strutture Dati**

Un programma è la combinazione di come rappresentiamo i dati e di come definiamo le operazioni che li elaborano.

Algoritmo per moltiplicazione tra interi

Scriviamo ora l'algoritmo di moltiplicazione tra due interi **m** ed **n** ≥ 0 come una sequenza esplicita di passi basata su somme ripetute.

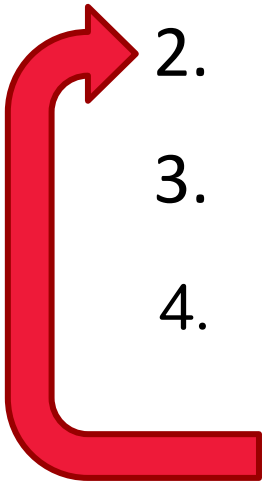
1. *Preliminarmente*, impostare al valore neutro 0 l'accumulatore *s*, ed impostare a 0 il contatore *i*.
2. Sommare *m* a *s* esattamente *n* volte, aggiornando il contatore *i* per tenere traccia del numero di addizioni effettuate, e contemporaneamente aggiornare l'accumulatore *s* per tenere traccia del risultato parziale

Punto 2 semanticamente complesso.

Algoritmo per moltiplicazione tra interi

Miglioriamo la formulazione per l'algoritmo di moltiplicazione tra due interi **m** ed **n** ≥ 0 come una sequenza esplicita di passi basata su somme ripetute.

1. *Preliminarmente*, imposta al valore neutro 0 l'accumulatore s, ed imposta a 0 il contatore *i*.
2. Somma *m* all'accumulatore *s*
3. Somma 1 al contatore *i*
4. Se *i* = *n* allora il numero di addizioni richieste è stato effettuato e *s* corrisponde con al totale, altrimenti ripetere il punto 2



Verifica dell'algoritmo di moltiplicazione

- Abbiamo sviluppato un algoritmo che risolve la moltiplicazione tra interi attraverso quattro punti in ordine dato.
- L'algoritmo è stato sviluppato a partire da una coppia di valori «di prova» di m ed n pari a m=4 ed n=3
- L'algoritmo ritorna un risultato corretto anche per un input diverso?
- Verifichiamo il funzionamento per un caso particolare dove n=0 per cui il risultato atteso è 0 (m x 0 = 0)

Verifica dell'algoritmo di moltiplicazione

- Esempio per *m=4 e n = 0*
- Risultato atteso s = 0

`s=0` `i=0`

Verifica dell'algoritmo di moltiplicazione

- Esempio per *m=4* e *n = 0*
- Risultato atteso s = 0
- Dopo la 1° addizione, *s* vale 4 ed *i* viene incrementato ad 1
- La condizione i = n non sarà mai verificata!
- L'algoritmo non termina mai!

```
   0  +
   4  =
------
 s=4  +      i=1
```



Analisi dell'algoritmo di moltiplicazione

- Nella formulazione precedente, prima aggiorno *s* ed *i*, e dopo confronto il contatore *i* con il numero di addizioni *n*
- Se *n* è pari a 0, l'algoritmo non terminerà mai (sempre i>n)
- Dobbiamo quindi prima verificare che il contatore *i* sia inferiore ad *n*, e solo dopo eventualmente eseguire l'addizione
- Il paradigma di progetto degli algoritmi

  «*prima verificare le condizioni e solo dopo eseguire (la prima di una serie di) operazioni*»

  ci accompagnerà nel resto di questo insegnamento
- La gestione del caso iniziale è una tipica fonte di errori, anche in sede d'esame

Algoritmo per moltiplicazione tra interi

Scriviamo una terza formulazione dell'algoritmo di moltiplicazione fra numeri interi *m* ed *n* tramite somme ripetute.

- **[1]** *Preliminarmente*, imposta l'accumulatore s al valore neutro 0, ed imposta il contatore *i* a 0.
- **[2]** Se *i* = *n* allora il numero di addizioni richieste è stato effettuato e s corrisponde con al totale, quindi terminiamo. Altrimenti continuiamo.
- **[3]** Somma *m* all'accumulatore *s*
- **[4]** Somma 1 al contatore *i*
- **[5]** Ritorna alla riga 2.

Algoritmo per moltiplicazione tra interi

Introduciamo una notazione più formale:

- ← operatore di assegnamento di numero intero a variabile
- = operatore di confronto tra variabili intere

Quindi:

É troppo forte parlare di terminazione: potremmo voler eseguire altre istruzioni dopo la moltiplicazione. Riformuliamo.

- **[1]** $s \leftarrow 0,\ i \leftarrow 0$
- **[2]** Se $i = n$ l'algoritmo termina, altrimenti
- **[3]** $s \leftarrow s + m$ ▷ somma m ad s
- **[4]** $i \leftarrow i + 1$ ▷ incrementa di 1 il contatore i
- **[5]** ritorna alla riga 2

Rientro lasciato di proposito per indicare che le operazioni sono condizionate alla valutazione della condizione alla riga **[2]**

Algoritmo per moltiplicazione tra interi

Quindi:

- **[1]** $s \leftarrow 0,\ i \leftarrow 0$
- **[2]** Se $i = n$ vai alla riga 6, altrimenti
  - **[3]** $s \leftarrow s + m$
  - **[4]** $i \leftarrow i + 1$
  - **[5]** **salta** alla riga 2
- **[6]** **Fine**

- Introduciamo quindi una speciale istruzione **Fine** per terminare l'algoritmo.
- Usiamo il **salto** verso un'istruzione numerata per ripetere passi o terminare.
- Riformuliamo ancora una volta l'algoritmo usando questa convenzione.

Algoritmo per moltiplicazione tra interi

Quindi:

- **[1]** $s \leftarrow 0,\ \ i \leftarrow 0$
- **[2]** Se $i = n$ allora **salta** alla riga 6, altrimenti
  - **[3]** $s \leftarrow s + m$
  - **[4]** $i \leftarrow i + 1$
  - **[5]** **salta** alla riga 2
- **[6]** Fine

- Per uniformità delimitiamo i blocchi con delle coppie di istruzioni speciali Inizio/Fine.
- Riformuliamo di nuovo.

Algoritmo per moltiplicazione tra interi

Quindi:

- **[1]** Inizio Algoritmo
- **[2]** $s \leftarrow 0,\ i \leftarrow 0$
- **[3]** Inizio Ripetizione Condizionata

  Se $i = n$ allora **salta** alla riga 7, altrimenti
  - **[4]** $s \leftarrow s + m$
  - **[5]** $i \leftarrow i + 1$
  - **[6]** **salta** alla riga 3
- **[7]** Fine Ripetizione Condizionata
- **[8]** Fine Algoritmo

Salto **condizionato**

Salto **non condizionato**

Algoritmo per moltiplicazione tra interi

La **struttura** ci permette di <u>eliminare</u> i numeri di riga:

- Inizio Algoritmo
  - $s \leftarrow 0,\ i \leftarrow 0$
  - Inizio Ripetizione Condizionata
    Se $i = n$ **salta** alla Fine Ripetizione Condizionata, altrimenti
    - $s \leftarrow s + m$
    - $i \leftarrow i + 1$
    - **salta** all'Inizio Ripetizione Condizionata
  - Fine Ripetizione Condizionata
- Fine Algoritmo

I **linguaggi di programmazione strutturati** organizzano le istruzioni in **blocchi annidabili**; l'indentazione rende visibile questa struttura gerarchica e l'appartenenza di ogni istruzione al proprio **blocco**.

Terminologia

Programma: descrizione di un algoritmo attraverso un *linguaggio di programmazione*

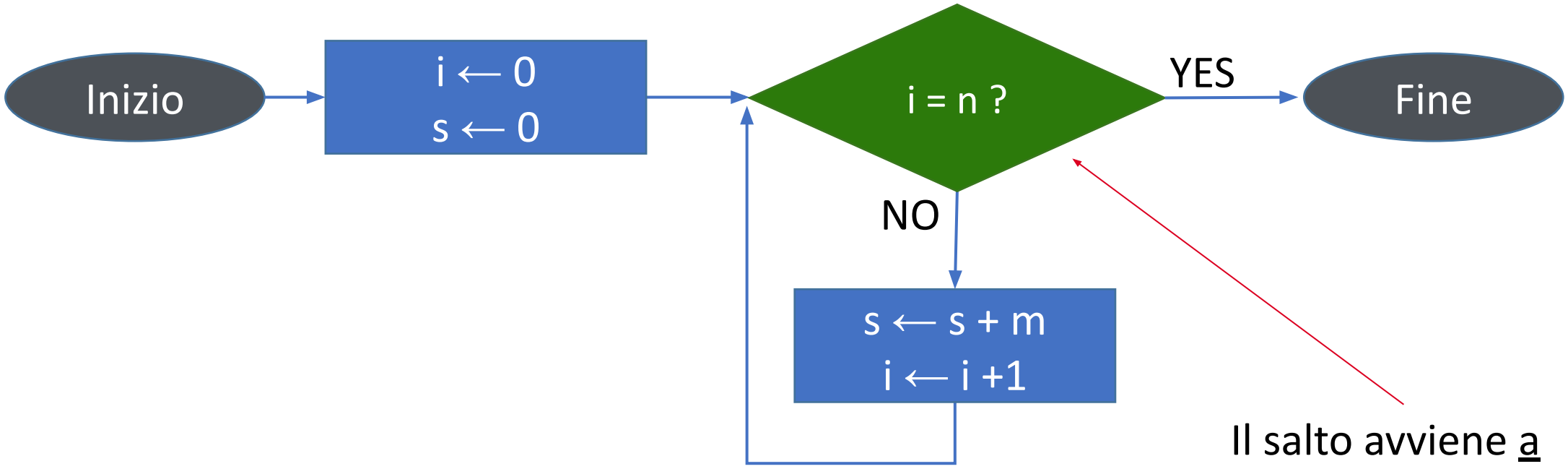
Linguaggio di programmazione:

- insieme di parole e di regole, definite in modo formale, per consentire la programmazione di un calcolatore affinché esegua compiti predeterminati
- I linguaggi di programmazione sono:
  - di basso livello *(es. linguaggio macchina, assembly, etc.)*
  - di alto livello vicini al linguaggio naturale *(es. FORTRAN, C, Java, Python, etc.)*

"Programma" per moltiplicazione tra interi

- Prima di passare al linguaggio di programmazione vero e proprio, introduciamo ancora una rappresentazione intermedia: il diagramma di flusso (*flow-chart*).
- Il flow-chart mette in evidenza il flusso di controllo, mostrando graficamente le possibili sequenze di esecuzione tra le istruzioni.
- A differenza della **rappresentazione strutturata** vista prima, il flow-chart non rende esplicito l'annidamento dei blocchi, ma segue direttamente i collegamenti tra le istruzioni.
- È una forma di rappresentazione storicamente molto usata e talvolta utile per visualizzare il controllo; i linguaggi moderni preferiscono però esprimere direttamente strutture di controllo annidate.

"Programma" per moltiplicazione tra interi



Il salto avviene a condizione che il contatore *i* sia uguale ad *n*

Il salto avviene senza condizioni

“Programma” per moltiplicazione tra interi



- Abbiamo un’idea di come implementare* il nostro algoritmo di moltiplicazione su un vero computer
- Lo scopo di questo corso è imparare ad implementare algoritmi elementari (molto semplici) nel linguaggio C, ovvero
- Tradurre** degli algoritmi specificati in Italiano in C
- Ci interessa quindi studiare un modello di calcolo che può essere eseguito su di un calcolatore meccanico o elettronico: la *macchina di Von Neumann*.

** realizzare concretamente una procedura usando un linguaggio di programmazione, partendo dalla sua definizione logica.*

*** nell'esame di Algoritmi lo scopo sarà invece quello di analizzare e progettare nuovi algoritmi.*